Algorithms and implementations

Mexico finals team reference

Built from the 2017-2025 regional research and pol-cova/kactl. A companion to the existing KACTL notebook, with explanations, contracts, and tested C++17 helpers.

Use the recognition tables first. Check assumptions before typing. All helper indices are zero-based unless explicitly stated. Advanced KACTL entries are navigation references; consult their source and listed dependencies. The team must confirm event material limits before choosing what to print.

Edition 1 - 18 October 2026
Editable sources and tests: handbooks/ in the repository.
This document does not replace the original kactl.pdf.

Contents and code lookup

Implementation handbook for the Mexico final 4

One-minute technique lookup . . . 4
Before copying code . . . 4
Components and ordinary BFS . . . 5
Grids and implicit state graphs . . . 6
Weighted shortest paths . . . 6
Dependencies, topological order, and DAG DP . . . 7
Connectivity changes and minimum spanning trees . . . 7
Trees without recursive DFS . . . 7
Ordered information and interval techniques . . . 8
Fenwick tree . . . 9
Segment tree . . . 9
From a slow solution to DP . . . 10
Worked DP: a huge number becomes a small remainder . . . 11
Code conventions and includes . . . 11
Remainder DP with lexicographic reconstruction . . . 12
Submission checks that catch real failures . . . 12
Extension map into the original KACTL . . . 12
Authoritative core C++17 code . . . 13
Code conventions and includes . . . 13
Unweighted BFS and path reconstruction . . . 14
Connected components . . . 14
Grid BFS . . . 15
Weighted distances: 0-1 BFS and Dijkstra . . . 15
Topological order . . . 16
Disjoint sets and minimum spanning forests . . . 16
Tree order, subtree sizes, and LCA . . . 17
Coordinate compression and prefix sums . . . 17
Difference arrays . . . 18
Sliding window for nonnegative sums . . . 18
Monotone binary search . . . 18
Fenwick sums and order statistics . . . 18
Iterative segment tree . . . 19
Capacity DP: 0-1 and unbounded knapsack . . . 19
Strict LIS with reconstruction . . . 19

Specialist lookup: choose the model before the template 20

Fast lookup . . . 21
Strings: structure before hashing . . . 22
Directed constraints and undirected connectivity . . . 23
Matching and capacity models . . . 23
Trees: path conventions are part of the algorithm . . . 24

Geometry: distinguish predicates from constructed coordinates . . . 24
Sweeps and optimized transitions . . . 24
Matrices and transforms: specialist fallback . . . 25
Printed source pack . . . 25
KACTL code: KMP . . . 26
KACTL code: Zfunc . . . 26
KACTL code: AhoCorasick . . . 26
KACTL code: SCC . . . 27
KACTL code: 2sat . . . 28
KACTL code: Dinic . . . 29
KACTL code: HopcroftKarp . . . 29
KACTL code: BinaryLifting . . . 30
KACTL code: Point . . . 30
KACTL code: OnSegment . . . 31
KACTL code: SegmentIntersection . . . 31
KACTL code: ConvexHull . . . 31
KACTL code: PolygonArea . . . 31
KACTL code: LineContainer . . . 32
KACTL code: Matrix . . . 32

Implementation handbook for the Mexico final

Use this book to turn an identified technique into correct C++17 code. The math companion explains arithmetic, counting, and proofs. This book explains state, containers, transitions, and the details that usually cost a submission.

The regional research motivates the selection. Graph modeling, range information, DP, and implementation discipline recur across the analyzed sets. A graph label can mean a difficult reduction, so a BFS template alone does not solve every problem in that category. Start by identifying what a vertex and an edge mean.

The code appendix is the authoritative `include/core.hpp`. Copy only the modules you need, together with their type aliases and standard includes. Every module lives in `namespace finals_core`. The original KACTL templates remain unchanged.

One-minute technique lookup

Need	First choice	Check before coding
Which vertices are connected?	Components or DSU	Undirected graph? Offline unions only?
Fewest moves, every move costs one	BFS	Is one vertex enough to describe a state?
Nearest of many starting positions	Multi-source BFS	Put every source at distance zero
Movement costs zero or one	0-1 BFS	Every edge weight must be 0 or 1
Minimum nonnegative total cost	Dijkstra	Negative edges invalidate it
Precedence, dependencies, DAG DP	Kahn topological order	Reject cycles before using DAG transitions
Cheapest way to connect an undirected graph	Kruskal + DSU	Report disconnected input explicitly
Repeated ancestor/path-length queries	Binary lifting LCA	Input must be a tree
Immutable interval sums	Prefix sums	Use 64-bit totals
Offline interval additions	Difference array	Materialize after all additions
Point changes, interval sums/counts	Fenwick tree	Updates are additions, intervals are half-open
Point assignment, interval min/max/custom merge	Segment tree	Associative merge and correct identity
Contiguous window with monotone feasibility	Two pointers	Negative values can break monotonicity
Minimum feasible answer	Binary search on the answer	Prove predicate monotonicity
Each item can be used once	0-1 knapsack	Descend capacities
Each item can be reused	Unbounded knapsack	Ascend capacities; positive weights
Longest strictly increasing subsequence	LIS + lower_bound	Equal values must not extend strict LIS

Before copying code

All vertex and array indices are zero-based. All array ranges are `[l,r)`, so `r` is excluded. Convert a one-based inclusive input `[L,R]` into `[L-1,R)`. An empty range has `l == r`. Prefix arrays have length `n+1`.

`ll` means `long long`. `INF = LLONG_MAX / 4` is an unreachable sentinel for weighted shortest paths. Every true shortest distance must be less than `INF`. For other sums, including MST weight, prefix sums, DP values, and products, prove that intermediate values fit. The sentinel does not automatically protect other arithmetic. When necessary, multiply with `__int128` before reducing or comparing.

Graph endpoints, source indices, and grid dimensions must be valid. Assertions catch some mistakes during practice; they are not input validation. Most graph functions assume `n` fits `int`, and the grid version also needs `height*width` to fit `int`. An empty graph is allowed only where no source/root is required.

A minimal caller is:

```
#include <iostream>
#include "include/core.hpp"
```

```
using namespace finals_core;
int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    Graph g(n);
    while (m--) {
        int u, v; std::cin >> u >> v; --u; --v;
        g[u].push_back(v);
        g[v].push_back(u); // Delete this line for directed input.
    }
    auto result = bfs(g, {0});
    std::cout << result.dist[n-1] << '\n';
}
```

Do not mix `scanf` and `cin` after disabling stream synchronization. Read the whole statement before deciding whether test cases have a leading count, end at EOF, or stop at a sentinel. Reset graph and DP storage between test cases.

Components and ordinary BFS

Connected components

`components(g)` traverses an undirected graph without recursion. It returns:

- `count`, the number of components;
- `id[u]`, a component label between zero and `count-1`;
- `representative[c]`, one vertex from component `c`;
- `size[c]`, the number of vertices in component `c`.

Time and extra storage are `O(n+m)` and `O(n)`. Each undirected edge normally appears twice in the adjacency lists. Isolated vertices count as components. A loop or parallel edge does not change the component partition.

To connect every component using the minimum number of new roads:

```
auto c = components(g);
std::cout << std::max(0, c.count-1) << '\n';
for (int i = 1; i < c.count; ++i)
    std::cout << c.representative[0]+1 << ' '
              << c.representative[i]+1 << '\n';
```

There must be at least `c.count-1` new edges because one new edge can reduce the component count by at most one. The construction reaches that bound. If `g` is directed, this function is neither an SCC algorithm nor a weak-components algorithm. Build an undirected view for weak components; use KACTL `graph/SCC.h` for strongly connected components.

BFS and paths

`bfs(g,{s})` returns `Search{dist,parent}`. `dist[u]` is the fewest edges from `s` to `u`, or `-1` when unreachable. A parent is assigned when the vertex first enters the queue. Each source has parent `-1` and distance zero.

The queue processes all distance `d` vertices before distance `d+1` vertices. That is why the first discovery gives a shortest path. Mark on insertion, not on removal, or many neighbors may insert the same vertex.

`restore_path(result,t)` returns the vertices along one shortest path, including its source and `t`. It returns an empty vector if `t` is unreachable. The path length in edges is `path.size()-1`; a problem may ask for the number of vertices instead. BFS costs `O(n+m)`, and path restoration costs `O(path length)`.

```
auto s = bfs(g, {start});
auto path = restore_path(s, finish);
if (path.empty()) std::cout << "IMPOSSIBLE\n";
else {
    std::cout << path.size() << '\n';
    for (int u : path) std::cout << u+1 << ' ';
    std::cout << '\n';
}
```

The chosen path depends on adjacency order. It is not necessarily lexicographically smallest. BFS also works on directed graphs, following only the stored directions.

Many sources

`bfs(g,{s1,s2,...})` computes distance to the closest source in one traversal. Duplicate sources are ignored. An empty source list leaves everything unreachable. Its cost is `O(n+m+number of supplied sources)`.

Do not run BFS separately from every source when the question is distance to any source. Insert them all at distance zero. If the question asks for the nearest red source and nearest blue source separately, run two multi-source searches with the two different source sets.

For spreading hazards, compute hazard arrival times first. A player may enter `v` at time `t` only if the problem allows it, often `t < hazard[v]`. Simultaneous arrival is unsafe in many statements. This needs a constrained player BFS; calling the unconstrained grid helper does not enforce the rule.

Grids and implicit state graphs

`grid_bfs(grid,{{row,column},...},'#')` performs four-neighbor BFS. Rows must have equal lengths. Every character other than `'#'` is traversable, including `A`, `B`, and dots. A wall source is ignored. Sources must be valid cells.

Cell `(r,c)` has vertex ID `r*width+c`. Recover coordinates using `id/width` and `id%width`. The returned `Search` has `height*width` entries. Traversal costs `O(height*width)` time and memory without constructing adjacency lists.

```
auto s = grid_bfs(board, {{sr,sc}});
int target = tr*width+tc;
auto path = restore_path(s,target);
std::string moves;
for (int i = 1; i < int(path.size()); ++i) {
    int dr = path[i]/width-path[i-1]/width;
    int dc = path[i]%width-path[i-1]%width;
    moves += dr == 1 ? 'D' : dr == -1 ? 'U' : dc == 1 ? 'R' : 'L';
}
```

An empty `path` means no route. If start equals target, it contains one cell and `moves` is empty, which is a valid zero-move path. For eight-neighbor movement, change the displacement arrays and recheck corner-cutting rules.

The most common modeling error is storing too little state. If legality depends on key possession, parity, remaining wall breaks, or orientation, `(r,c)` alone is insufficient. Use `(r,c,keyMask)`, `(r,c,time%period)`, or the required state. Allocate distances per full state. Reaching the same cell with a different key set can lead to a different future.

Count the states before writing code. A grid with `h*w` cells and `2^k` key sets has up to `h*w*2^k` vertices. An implicit graph saves edge storage, not state count. Keep the original board separate from visited state when revisits with different state values are meaningful.

Weighted shortest paths

Both weighted helpers use `WGraph`, an adjacency list of pairs `{neighbor,weight}`. They return `WeightedSearch{dist,parent}`. Unreachable distance is `INF`, and a source parent is `-1`. To restore a path, first check `dist[t] != INF`, follow `parent` back to `-1`, and reverse. Do not pass a `WeightedSearch` directly to `restore_path`, which accepts the unweighted `Search` type.

0-1 BFS

`zero_one_bfs(g,s)` finds minimum total weight when every edge has weight zero or one. A zero-cost relaxation goes to the deque front; a one-cost relaxation goes to the back. This preserves distance order. Time is `O(n+m)`, memory is `O(n+m)` including queued entries.

```
WGraph g(n);
g[u].push_back({v,0}); // Free transition.
g[v].push_back({u,1}); // Paid reverse transition, if the model says so.
auto s = zero_one_bfs(g,start);
```

Unlike ordinary BFS, a discovered vertex can later receive a better distance. Use relaxation `newDistance < dist[v]`, not one permanent visited flag. A common application assigns cost one to reversing an edge and zero to following it. The statement must justify those transitions.

Dijkstra

`dijkstra(g,s)` accepts nonnegative `ll` edge weights. Time is `O((n+m) log(n+m+1))` with this lazy heap implementation; memory is `O(n+m)`. For a simple graph the usual bound is written `O((n+m) log n)`. A stale heap entry is discarded when its saved distance differs from `dist[u]`.

```
auto s = dijkstra(weighted,start);
if (s.dist[finish] == INF) std::cout << "IMPOSSIBLE\n";
else std::cout << s.dist[finish] << '\n';
```

Negative edges invalidate Dijkstra, even if there is no negative cycle. Use a suitable negative-edge algorithm, such as KACTL `graph/BellmanFord.h`, after checking its contract. For all-pairs distances with small `n`, inspect `graph/FloydWarshall.h`; its cubic time often suits `n` of a few hundred.

The implementation skips candidate distances at least `INF` before addition. This avoids overflowing that addition, but such paths cannot be represented. Choose a larger safe sentinel or wider arithmetic if constraints require it. Zero-weight cycles are fine. Counting shortest paths in their presence needs extra reasoning; adding a `ways` update to ordinary Dijkstra may be wrong.

Dependencies, topological order, and DAG DP

`topological_order(g)` uses Kahn's algorithm on a directed graph. It returns a permutation in which each edge `u -> v` places u before `v`. If a cycle exists, it returns an empty vector. Check `order.size() == g.size()`, which also handles the valid empty graph. Time and storage are `O(n+m)` and `O(n)`.

Use indegree zero as the initial queue. Reducing indegrees simulates removing completed tasks. A vertex can enter the queue only after all prerequisites have been removed. Parallel edges count separately and are removed separately.

A longest path from `source` in an unweighted DAG:

```
auto order = topological_order(g);
if (order.size() != g.size()) { /* Report a cycle; stop DAG DP. */ }
else {
    std::vector<ll> best(g.size(),-INF);
    std::vector<int> parent(g.size(),-1);
    best[source] = 0;
    for (int u : order) if (best[u] != -INF)
        for (int v : g[u]) if (best[u]+1 > best[v]) {
            best[v] = best[u]+1; parent[v] = u;
        }
    // best[target] == -INF means unreachable, not a zero-length path.
}
```

Replace `max` by modular addition for path counts, or by `min` for shortest paths on a DAG. DAG shortest paths can have negative weights because the topological order prevents revisiting a dependency. Parallel edges count as distinct paths unless you explicitly deduplicate them.

A recurrence is not automatically a DAG algorithm. First identify the dependency order and prove that every transition uses an already computed state.

Connectivity changes and minimum spanning trees

`DSU(n)` stores n initially separate sets. `find(u)` returns a representative, `unite(u,v)` returns true exactly when it merged two sets, and `size(u)` returns the size of the current set containing u. Path compression and union by size give amortized `O(alpha(n))` time per operation and `O(n)` memory.

DSU answers connectivity under additions. It does not give a graph path, a shortest distance, or support arbitrary edge deletions. The representative is an implementation detail and may change after a union.

`kruskal(n,edges)` sorts undirected `Edge{u,v,w}` values and accepts an edge whenever it joins different DSU sets. It returns `MST{weight,components,edges}`. For connected nonempty input, `components == 1` and exactly `n-1` edges are selected. For disconnected input it returns a minimum spanning forest.

```
std::vector<Edge> edges = {{0,1,4},{1,2,2},{0,2,9}};
auto mst = kruskal(3,edges); // weight 6, two selected edges.
if (mst.components != 1) { /* No spanning tree exists. */ }
```

Kruskal costs `O(m log m)` time plus near-linear DSU work. Negative edge weights, parallel edges, and self-loops are supported; a self-loop never joins two sets. Store each undirected edge once in the edge list. A minimum spanning tree minimizes the sum of selected edge weights, not all source-to-vertex distances. Dijkstra's shortest-path tree solves a different objective.

For offline deletions, sometimes process operations in reverse and turn deletions into additions. For rollback, see KACTL `data-structures/UnionFindRollback.h`. Check whether path compression is compatible with rollback before modifying it.

Trees without recursive DFS

`root_tree(tree,root)` assumes a nonempty connected undirected tree. It returns `parent`, `depth`, `order`, and `subtree`. The root is its own parent. `depth` counts edges from the root. `order` places parents before children; reversing it places children before parents. `subtree[u]` includes u itself.

Time and storage are `O(n)`. No recursive call can overflow the runtime stack. The function visits vertices only once, but on a graph with cycles it would produce a spanning-tree order rather than prove that the input is a tree. The caller must enforce the tree assumption.

A common tree DP asks for a maximum independent set, a largest set with no adjacent selected vertices:

```
auto info = root_tree(tree,0);
std::vector<ll> take(n,1), skip(n,0);
for (int i = n-1; i > 0; --i) {
    int u = info.order[i], p = info.parent[u];
    take[p] += skip[u];
    skip[p] += std::max(take[u],skip[u]);
}
ll answer = std::max(take[0],skip[0]);
```

The two states mean whether u itself is selected. If it is selected, children must be skipped. If it is skipped, each child chooses its better state. This works because different child subtrees have no cross edges.

This `order` is not an Euler tour and does not make each subtree a contiguous interval. For subtree range queries, explicitly compute DFS entry/exit times. For path range queries and updates, inspect KACTL `graph/HLD.h` after learning heavy-light decomposition; an ordinary Fenwick tree over the current `order` does not solve that task.

Binary lifting LCA

Construct `LCA ancestor(info)`. `jump(u,k)` returns the kth ancestor or `-1` when `k` exceeds `depth[u]`. `query(u,v)` returns the lowest common ancestor. `distance(u,v)` returns the number of edges on the unique tree path. Preprocessing costs `O(n log n)` time and storage; each query costs `O(log n)`.

```
auto info = root_tree(tree,root);
LCA ancestor(info);
int common = ancestor.query(u,v);
int edgesOnPath = ancestor.distance(u,v);
```

The table entry `up[k][u]` is the ancestor `2^k` steps above `u`. First lift the deeper vertex to equal depth. Then lift both from the largest powers down while their ancestors differ. Their immediate parent is the LCA.

For a weighted tree, compute `rootDistance[u]` separately and use `rootDistance[u] + rootDistance[v] - 2*rootDistance[common]`. The provided `distance` method counts edges and does not know edge weights. For forests, add component IDs and reject cross-component queries before using a suitable forest-aware implementation.

Ordered information and interval techniques

Coordinate compression

`compressed_values(a)` returns the sorted distinct values. `rank_of(values,x)` returns the lower-bound position of `x`. It is an exact compressed index only when `x` belongs to `values`; otherwise it is an insertion point, possibly `values.size()`. Build values from every coordinate that later needs an index. Sorting costs `O(n log n)`, and each lookup costs `O(log n)`.

Compression preserves order and equality, not numerical distances. Values `10` and `1000000` can become adjacent ranks. Recover real gaps from the original coordinate vector when computing lengths or areas.

```
std::vector<ll> a = {40,10,40,90};
auto values = compressed_values(a); // 10,40,90.
Fenwick counts(int(values.size()));
for (ll x : a) counts.add(rank_of(values,x),1);
```

To count inversions, scan left to right. The number of earlier values greater than `a[i]` is `seen - counts.prefix(rank+1)`. Then add one at `rank`. Querying before updating avoids counting the current element.

Prefix sums

`prefix_sums(a)` returns `p[0]=0` and `p[i+1]=p[i]+a[i]`. Thus the sum on `[l,r)` is `p[r]-p[l]`, including empty ranges. Preprocessing costs `O(n)`, a query costs `O(1)`, and storage is `O(n)`. Negative entries are supported.

Do not rebuild prefix sums after every update. If updates are online, use a Fenwick or segment tree. For immutable rectangles, use a two-dimensional prefix sum and inclusion-exclusion; KACTL `data-structures/SubMatrix.h` is a reference.

Difference arrays

Allocate `std::vector<ll> diff(n+1)`. `difference_add(diff,l,r,x)` adds `x` to all positions in `[l,r)`. After all updates, `materialize_difference(diff)` returns the resulting length-n array. To update an existing array, add the materialized increments to it, or initialize `diff` with the differences of that array.

```
std::vector<ll> diff(6); // Five actual positions plus one sentinel.
difference_add(diff,1,4,7);
difference_add(diff,3,5,-2);
auto a = materialize_difference(diff); // 0,7,7,5,-2.
```

Each update costs `O(1)`; materialization costs `O(n)`. The sentinel at `diff[n]` is necessary for updates ending at `n`. A difference array supports offline updates, followed by queries. Interleaved updates and queries require another data structure.

Two pointers and sliding windows

`count_subarrays_at_most(a,limit)` counts nonempty contiguous subarrays whose sum is at most `limit`, assuming all entries are nonnegative. It costs `O(n)` time and `O(1)` extra space. Zeros are valid. A negative limit returns zero.

At each right endpoint, remove leftmost entries until the window is feasible. All starts between `l` and `r` are then feasible, so add `r-l+1`. Each pointer moves forward at most `n` times.

For exact sum `k` with nonnegative entries, use `atMost(k)-atMost(k-1)`. Negative entries destroy this window argument. For arbitrary signed entries and exact sum, count earlier prefix sums equal to `currentPrefix-k` using a map. Initialize the count of prefix zero to one. If you need deterministic bounds, choose `std::map` or compressed coordinates instead of assuming constant-time hashing.

Binary search on a monotone predicate

`first_true(lo,hi,ok)` searches the nonnegative integer interval `[lo,hi)`. The predicate must be false and then true as the argument increases. It returns the first true position, or the original `hi` if none exists. It never evaluates `ok(hi)`. Its cost is `O(log(hi-lo+1) * predicate cost)`.

```
ll answer = first_true(0,1001,[&](ll x) { return x*x >= 200; });
// answer == 15. Multiplication is safe for this small bound.
```

Choose an excluded upper endpoint that includes every candidate you care about. When `hi` is a proven feasible sentinel but intentionally excluded, returning `hi` still gives that answer. Record which interpretation you use.

Before coding, write a sentence such as "if capacity x works, every larger capacity works." If that sentence is false, answer binary search is invalid. A greedy feasibility test still needs its own proof. This helper's midpoint is safe for nonnegative bounds; do not pass negative search intervals to it.

Fenwick tree

`Fenwick(n)` starts with `n` zeros. `add(i,delta)` adds to index `i`. `prefix(r)` sums `[0,r)`, and `sum(l,r)` sums `[l,r)`. Updates and queries cost `O(log n)`; storage is `O(n)`. Constructing and adding `n` initial entries costs `O(n log n)` with this version.

```
Fenwick fw(n);
for (int i = 0; i < n; ++i) fw.add(i,a[i]);
ll interval = fw.sum(l,r);
// Assignment a[i] = value requires a delta:
fw.add(i,value-a[i]); a[i] = value;
```

Negative values are allowed for sums. They are not allowed for order statistics. `lower_bound(k)` requires `k > 0` and every represented entry nonnegative. It returns the zero-based index of the first position where the inclusive prefix reaches `k`, or `n` if the total is less than `k`.

For frequency counts `[0,2,0,3]`, the first and second elements in sorted order have index `1`; the third through fifth have index 3. Therefore `lower_bound(1)==1`, `lower_bound(3)==3`, and `lower_bound(6)==4`. This can remove the `k`th remaining item with `add(index,-1)`, provided its frequency stays nonnegative.

The internal array is one-based, while the public API is zero-based. Copy the whole module; mixing conventions is a common off-by-one failure. See KACTL `data-structures/FenwickTree.h` and `FenwickTree2d.h` for related templates.

Segment tree

`SegTree<T,Op>` supports point assignment and associative range aggregation. The constructor takes the initial array and an identity value. `set(i,value)` assigns one position; `query(l,r)` merges `[l,r)`. Build time and storage are `O(n)`; each update and query costs `O(log n)`.

```
SegTree<ll,Sum> sums(a,0LL);
SegTree<ll,Minimum> mins(a,std::numeric_limits<ll>::max());
sums.set(2,8);
ll total = sums.query(1,4);
ll smallest = mins.query(1,4); // Independent tree; still has original a[2].
```

The merge must be associative: `op(op(a,b),c) == op(a,op(b,c))`. It need not be commutative because the query preserves order. The identity must satisfy `op(identity,x) == op(x,identity) == x`. For sums use zero, minimum

use the largest representable value, maximum use the smallest representable value, and GCD use zero. Empty queries return the identity.

For maximum subarray sum, each node needs total sum, best prefix, best suffix, and best subarray. Define exactly whether an empty subarray is allowed, then choose the node identity and merge rules consistently. A tree of scalar sums alone cannot answer that query.

This implementation does not support interval updates. Repeating point updates for every element can time out. For range addition/range maximum, or range assignment/range sums, use lazy propagation and explicitly define how a pending operation composes with another. KACTL `data-structures/LazySegmentTree.h` has a specific contract; read it before adapting. Its update semantics are not implied by this point-assignment API.

From a slow solution to DP

A DP state records enough information that the future no longer depends on the full history. Write these six lines before selecting an array size:

- What does one state mean, in one exact sentence?
- Which decisions enter or leave this state?
- What are the base states and impossible-state values?
- In what order are all dependencies already known?
- What is the number of states times transitions per state?
- Which state contains the answer, and do you need a witness?

For example, brute force tries every subset of items. Many histories reach the same pair `(items considered,capacity remaining)`. Keep the best value for that pair; histories with lower value and the same future options can be discarded. That gives `O(n*capacity)` DP rather than `2^n` enumeration. If capacity is huge, this is not an efficient solution merely because it is DP. Consider DP by value, meet in the middle, a smaller mathematical state, or another technique.

For exact reachability or minimum cost, initialize impossible states explicitly. Examples are `false`, `INF`, or `-INF` depending on the operation. Initializing every exact-sum state to zero falsely makes unreachable sums reachable. The provided knapsack functions use an "at most capacity" objective, so zero is valid everywhere because taking no items is allowed.

0-1 knapsack

`knapsack01(weight,value,capacity)` returns the maximum value using each item at most once and total weight at most capacity. The vectors must have equal length, weights are nonnegative integers, and capacity is nonnegative. Time is `O(number of items * capacity)` and space is `O(capacity)`.

```
ll best = knapsack01({2,3,4},{4,5,7},5); // 9, using weights 2 and 3.
```

Descending capacity ensures `dp[c-weight]` still describes the previous item layer. Ascending would allow the current item to be reused. Zero-weight items are supported here because every capacity is updated once per item. Negative values are allowed; choosing no item gives a result of at least zero.

If you need the selected items, a parent pointer stored in the overwritten one-dimensional array is not automatically safe. Later updates can change a parent state and create an invalid witness. Store a full two-dimensional decision table, immutable predecessor records, or use a proven reconstruction method. Budget the memory before choosing the first option.

Unbounded knapsack and counting loop order

`knapsack_unbounded` has the same objective but permits unlimited copies of every item. Weights must be strictly positive. Capacity goes upward so that the current item can extend an already updated state. A positive-value zero-weight item would make the optimum unbounded and is rejected by this API.

```
ll best = knapsack_unbounded({2,3},{4,5},6); // 12, three copies of weight 2.
```

Counting is different from maximizing. To count unordered coin combinations, put the coin loop outside and ascend sums. To count ordered sequences of coin choices, put the target sum outside and sum over each possible last coin. For coins `{1,2}` and total 3, combinations are `{1,1,1}` and `{1,2}`, while ordered sequences are `(1,1,1)`, `(1,2)`, `(2,1)`. Duplicate denominations need attention; they may be distinct item types or accidental repeated values.

Longest increasing subsequence

`lis_indices(a)` returns indices of one longest strictly increasing subsequence, in their original order. It costs `O(n log n)` time and `O(n)` space. Values may be repeated or negative. Empty input returns an empty vector.

```
std::vector<int> a = {3,1,2,2,4};
auto ids = lis_indices(a); // Length 3, with values 1,2,4.
for (int i : ids) std::cout << a[i] << ' ';
```

`tails[k]` is the smallest possible last value for a strictly increasing subsequence of length `k+1`. It is not itself necessarily a subsequence from the input. The separate predecessor indices reconstruct an actual witness. To obtain a longest nondecreasing subsequence, change `lower_bound` to `upper_bound` and keep the predecessor logic consistent. See KACTL `various/LIS.h` for its own indexing and reconstruction contract.

Worked DP: a huge number becomes a small remainder

Suppose a digit pattern contains fixed digits and '?', and you need the smallest completion divisible by m. Trying every completion costs up to 10^q, where q is the number of unknown digits. Two prefixes at the same position with the same remainder have identical future options. Merge them.

Define `can[pos][r]`: after reading pos digits with remainder r, some permitted suffix can finish with remainder zero. Base: `can[length][0]=true`, every other terminal remainder is false. Transition: try allowed digit d and ask `can[pos+1][(10*r+d)%m]`. Fill positions backwards. Leading-zero restrictions apply at position zero, including fixed digits.

To reconstruct, start at pos=0,r=0. Try digits in increasing order and take the first whose next state is feasible. This is lexicographically smallest because a larger first differing digit can never improve the number, and feasibility guarantees that the chosen prefix can finish. Fixed-length canonical numbers have the same lexicographic and numeric order.

Example `?2`, modulus7: candidates 12,22,32,42,... first succeed at42. Looking only for whether any completion exists would not by itself produce this reconstruction.

The helper `finals_dp::complete_digits("?2",7)` returns an optional string containing `"42"`. `nullopt` means impossible. It uses O(length*m*10) time and O(length*m) bytes plus vector overhead. For length1000,m1000, there are roughly one million stored states and at most ten million tested transitions. Estimate this product before allocating. Counts-only DP can roll two rows, but reconstruction needs feasibility information or a different recovery method. The helper is for decimal pattern completion, not all digit-DP tasks: upper-bound tightness, digit sums, or forbidden substrings require extra state.

Counting loop order changes the objects

For positive coin values with unlimited reuse, initialize `ways[0]=1` and other entries zero. To count unordered multisets, process each coin type outside and sums ascending inside. To count ordered sequences, process sums ascending outside and last coin inside. With coins1,2 and target3, unordered answers are 1+1+1 and1+2, hence2. Ordered answers also include2+1, hence3. Duplicate denominations count as distinct types unless deduplicated. Reduce additions modulo the requested modulus or prove the count fits.

Small-subset assignment DP

When n is around20 and the state is the chosen subset, `dp[mask]` can store the minimum cost of assigning the first popcount(mask) workers to the jobs in mask. For every unchosen job j, relax `dp[mask | (1<<j)]` with `dp[mask]+cost[popcount(mask)][j]`. Set dp[0]=0 and others infinity; skip unreachable masks. Time O(n*2^n), memory O(2^n). This state is sufficient because only which jobs remain affects later workers. It is not a general substitute for polynomial matching on larger instances. Check shift widths and memory before computing `1<<n`.

Copyable remainder DP

This separate header has no dependency on core.hpp.

Code conventions and includes

```
#pragma once
#include <optional>
#include <stdexcept>
#include <string>
#include <vector>
namespace finals_dp {
```

Remainder DP with lexicographic reconstruction

Source: include/digit_dp.hpp | SHA256 fragment 78c5cd2a

```
// Nonempty pattern of digits and '?', positive modulus; indices fit int.
// smallest valid completion divisible by modulus, or nullopt.
// With forbid_leading_zero, even a one-digit zero is forbidden.
// Time O(length * modulus * 10), memory O(length * modulus).
inline std::optional<std::string> complete_digits(const std::string& pattern,
        int modulus, bool forbid_leading_zero = true) {
    if (pattern.empty() || modulus <= 0)
        throw std::invalid_argument("nonempty pattern and positive modulus required");
    for (char c : pattern)
        if (c != '?' && (c < '0' || c > '9'))
            throw std::invalid_argument("pattern must contain digits or question marks");
    int n = int(pattern.size());
    std::vector<std::vector<unsigned char>> can(n+1,
        std::vector<unsigned char>(modulus,0));
    can[n][0] = 1;
    auto allowed = [&](int pos, int digit) {
        return !(forbid_leading_zero && pos == 0 && digit == 0)
            && (pattern[pos] == '?' || pattern[pos] == char('0'+digit));
    };
    for (int pos = n-1; pos >= 0; --pos)
        for (int rem = 0; rem < modulus; ++rem)
            for (int d = 0; d <= 9; ++d) if (allowed(pos,d)) {
                int next = int((10LL*rem+d)%modulus);
                if (can[pos+1][next]) { can[pos][rem] = 1; break; }
            }
    if (!can[0][0]) return std::nullopt;
    std::string answer; int rem = 0;
    for (int pos = 0; pos < n; ++pos)
        for (int d = 0; d <= 9; ++d) if (allowed(pos,d)) {
            int next = int((10LL*rem+d)%modulus);
            if (can[pos+1][next]) {
                answer += char('0'+d); rem = next; break;
            }
        }
    return answer;
}
} // namespace finals_dp
```

Submission checks that catch real failures

Use one tiny normal case, one smallest case, and one case that attacks the assumption behind the technique. For BFS, include an isolated target and a cycle. For Dijkstra, include a cheaper indirect route and a zero-weight edge. For interval code, query `[0,n)`, `[0,0)`, and `[n,n)`. For knapsack, use one item that fits twice to detect an accidental unbounded update.

Check whether the answer counts edges or vertices, whether the source itself is an allowed answer, and whether equal values are strict or non-strict. Disconnected graphs, repeated coordinates, zero values, and multiple test cases usually expose more bugs than a large random example viewed by eye.

Compile in the judge's language mode. During practice add sanitizers and warnings when available. The accompanying `tests/core_test.cpp` compares the modules against small brute-force oracles; it does not prove that a new problem has been modeled correctly. Write down the reduction and its assumptions before trusting a familiar template.

Extension map into the original KACTL

These are references for tasks beyond this companion. Check the original header's description, dependencies, and indexing before use. Advanced templates require practice before the contest.

Need	Original repository file	Added responsibility
Strongly connected directed components	`content/graph/SCC.h`	Reduce the condensation graph to a DAG
Boolean implications	`content/graph/2sat.h`	Encode both implications correctly
Euler trail/circuit	`content/graph/EulerWalk.h`	Verify degree and connectivity conditions
Bridges and biconnected structure	`content/graph/BiconnectedComponents.h`	Distinguish vertices from edges in the result
Bipartite matching	`content/graph/HopcroftKarp.h`	Identify the two partitions and allowed pairs

Need	Original repository file	Added responsibility
Capacity flow	`content/graph/Dinic.h`	Construct capacities, source, sink, and prove reduction
Paths with costs and capacities	`content/graph/MinCostMaxFlow.h`	Read negative-cost and potential assumptions
Online subtree/path operations	`content/graph/HLD.h`	Choose correct node/edge intervals
Static range minimum	`content/data-structures/RMQ.h`	Updates are not supported
Range updates	`content/data-structures/LazySegmentTree.h`	Match lazy composition to operation semantics
Dynamic line minimum/maximum	`content/data-structures/LineContainer.h`	Verify slope and arithmetic conventions
DP optimization	`content/various/DivideAndConquerDP.h`, `KnuthDP.h`	Prove the required monotonicity first

Matching, flow, string algorithms, and computational geometry have additional material in the other companions or original KACTL. Do not interpret this core as complete coverage of every regional problem.

Authoritative core C++17 code

The following listing is the tested header used by the examples and randomized checks. The `BOOK` markers divide it into lookup sections in the printed build.

Code conventions and includes

```
#pragma once
#include <algorithm>
#include <cassert>
#include <deque>
#include <functional>
#include <limits>
#include <numeric>
#include <queue>
#include <string>
#include <utility>
#include <vector>

namespace finals_core {
using ll = long long;
using Graph = std::vector<std::vector<int>>;
using WGraph = std::vector<std::vector<std::pair<int, ll>>>;
constexpr ll INF = std::numeric_limits<ll>::max() / 4;
```

Unweighted BFS and path reconstruction

Source: include/core.hpp | SHA256 fragment 7a57f7b0

```
struct Search {
    std::vector<int> dist, parent;
};
inline Search bfs(const Graph& g, const std::vector<int>& sources) {
    int n = int(g.size());
    Search a{std::vector<int>(n, -1), std::vector<int>(n, -1)};
    std::queue<int> q;
    for (int s : sources) if (a.dist[s] == -1) {
        a.dist[s] = 0; q.push(s);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (a.dist[v] == -1) {
            a.dist[v] = a.dist[u] + 1;
            a.parent[v] = u; q.push(v);
        }
    }
    return a;
}
inline std::vector<int> restore_path(const Search& a, int target) {
    if (a.dist[target] == -1) return {};
    std::vector<int> p;
    for (int u = target; u != -1; u = a.parent[u]) p.push_back(u);
    std::reverse(p.begin(), p.end());
    return p;
}
```

Connected components

Source: include/core.hpp | SHA256 fragment 3f610dcc

```
struct Components {
    int count = 0;
    std::vector<int> id, representative, size;
};
inline Components components(const Graph& g) {
    Components a;
    a.id.assign(g.size(), -1);
    for (int s = 0; s < int(g.size()); ++s) if (a.id[s] == -1) {
        int id = a.count++;
        a.representative.push_back(s); a.size.push_back(0);
        std::vector<int> st{s}; a.id[s] = id;
        while (!st.empty()) {
            int u = st.back(); st.pop_back(); ++a.size[id];
            for (int v : g[u]) if (a.id[v] == -1) {
                a.id[v] = id; st.push_back(v);
            }
        }
    }
    return a;
}
```

Grid BFS

Source: include/core.hpp | SHA256 fragment 88e05248

```
// Four-neighbor grid; every character other than wall is traversable.
inline Search grid_bfs(const std::vector<std::string>& grid,
                const std::vector<std::pair<int, int>>& sources,
                char wall = '#') {
    int h = int(grid.size()), w = h ? int(grid[0].size()) : 0;
    Search a{std::vector<int>(h*w, -1), std::vector<int>(h*w, -1)};
    std::queue<int> q;
    for (auto [r,c] : sources) {
        assert(0 <= r && r < h && 0 <= c && c < w);
        if (grid[r][c] != wall && a.dist[r*w+c] == -1) {
            a.dist[r*w+c] = 0; q.push(r*w+c);
        }
    }
    const int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
    while (!q.empty()) {
        int u = q.front(); q.pop();
        int r = u/w, c = u%w;
        for (int k = 0; k < 4; ++k) {
            int nr = r+dr[k], nc = c+dc[k];
            if (nr < 0 || nr >= h || nc < 0 || nc >= w) continue;
            int v = nr*w+nc;
            if (grid[nr][nc] == wall || a.dist[v] != -1) continue;
            a.dist[v] = a.dist[u]+1; a.parent[v] = u; q.push(v);
        }
    }
    return a;
}
```

Weighted distances: 0-1 BFS and Dijkstra

Source: include/core.hpp | SHA256 fragment 914f1623

```
struct WeightedSearch {
    std::vector<ll> dist;
    std::vector<int> parent;
};
inline WeightedSearch zero_one_bfs(const WGraph& g, int source) {
    int n = int(g.size());
    WeightedSearch a{std::vector<ll>(n, INF), std::vector<int>(n,-1)};
    std::deque<std::pair<ll,int>> q;
    a.dist[source] = 0; q.push_front({0,source});
    while (!q.empty()) {
        auto [du,u] = q.front(); q.pop_front();
        if (du != a.dist[u]) continue;
        for (auto [v,w] : g[u]) {
            assert(w == 0 || w == 1);
            if (du+w < a.dist[v]) {
                a.dist[v] = du+w; a.parent[v] = u;
                if (w == 0) q.push_front({du,v});
                else q.push_back({du+1,v});
            }
        }
    }
    return a;
}
inline WeightedSearch dijkstra(const WGraph& g, int source) {
    int n = int(g.size());
    WeightedSearch a{std::vector<ll>(n, INF), std::vector<int>(n,-1)};
    using Node = std::pair<ll,int>;
    std::priority_queue<Node,std::vector<Node>,std::greater<Node>> q;
    a.dist[source] = 0; q.push({0,source});
    while (!q.empty()) {
        auto [du,u] = q.top(); q.pop();
        if (du != a.dist[u]) continue;
        for (auto [v,w] : g[u]) {
            assert(w >= 0);
            if (w >= INF-du) continue;
            if (du+w < a.dist[v]) {
                a.dist[v] = du+w; a.parent[v] = u; q.push({du+w,v});
            }
        }
    }
    return a;
}
```

Topological order

Source: include/core.hpp | SHA256 fragment f8d5236a

```
// Empty result means a cycle unless the input graph is empty.
inline std::vector<int> topological_order(const Graph& g) {
    int n = int(g.size());
    std::vector<int> deg(n), order;
    for (const auto& adj : g) for (int v : adj) ++deg[v];
    std::queue<int> q;
    for (int u = 0; u < n; ++u) if (!deg[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--deg[v] == 0) q.push(v);
    }
    if (int(order.size()) != n) return {};
    return order;
}
```

Disjoint sets and minimum spanning forests

Source: include/core.hpp | SHA256 fragment ef123988

```
struct DSU {
    std::vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n,1) { std::iota(p.begin(),p.end(),0); }
    int find(int u) {
        while (u != p[u]) { p[u] = p[p[u]]; u = p[u]; }
        return u;
    }
    bool unite(int u, int v) {
        u = find(u); v = find(v);
        if (u == v) return false;
        if (sz[u] < sz[v]) std::swap(u,v);
        p[v] = u; sz[u] += sz[v]; return true;
    }
    int size(int u) { return sz[find(u)]; }
};
struct Edge { int u,v; ll w; };
struct MST { ll weight = 0; int components = 0; std::vector<Edge> edges; };
inline MST kruskal(int n, std::vector<Edge> edges) {
    std::sort(edges.begin(),edges.end(),[](Edge a,Edge b){ return a.w < b.w; });
    DSU d(n); MST a; a.components = n;
    for (Edge e : edges) if (d.unite(e.u,e.v)) {
        a.weight += e.w; a.edges.push_back(e); --a.components;
    }
    return a;
}
```

Tree order, subtree sizes, and LCA

Source: include/core.hpp | SHA256 fragment 2594ba9e

```
struct TreeInfo {
    std::vector<int> parent, depth, order, subtree;
};
inline TreeInfo root_tree(const Graph& tree, int root = 0) {
    int n = int(tree.size());
    assert(n > 0 && 0 <= root && root < n);
    TreeInfo t{std::vector<int>(n,-1),std::vector<int>(n),{},std::vector<int>(n,1)};
    t.parent[root] = root; t.order.push_back(root);
    for (int i = 0; i < int(t.order.size()); ++i) {
        int u = t.order[i];
        for (int v : tree[u]) if (t.parent[v] == -1) {
            t.parent[v] = u; t.depth[v] = t.depth[u]+1; t.order.push_back(v);
        }
    }
    assert(int(t.order.size()) == n);
    for (int i = n-1; i > 0; --i) {
        int u = t.order[i]; t.subtree[t.parent[u]] += t.subtree[u];
    }
    return t;
}
struct LCA {
    std::vector<std::vector<int>> up;
    std::vector<int> depth;
    explicit LCA(const TreeInfo& t) : depth(t.depth) {
        int n = int(depth.size()), levels = 1;
        assert(n > 0);
        while ((1LL << levels) <= n) ++levels;
        up.assign(levels,std::vector<int>(n)); up[0] = t.parent;
        for (int k = 1; k < levels; ++k)
            for (int u = 0; u < n; ++u) up[k][u] = up[k-1][up[k-1][u]];
    }
    int jump(int u, int steps) const {
        if (steps < 0 || steps > depth[u]) return -1;
        for (int k = 0; k < int(up.size()); ++k) if ((steps >> k)&1) u = up[k][u];
        return u;
    }
    int query(int u, int v) const {
        if (depth[u] < depth[v]) std::swap(u,v);
        u = jump(u,depth[u]-depth[v]);
        if (u == v) return u;
        for (int k = int(up.size())-1; k >= 0; --k) if (up[k][u] != up[k][v]) {
            u = up[k][u]; v = up[k][v];
        }
        return up[0][u];
    }
    int distance(int u, int v) const { return depth[u]+depth[v]-2*depth[query(u,v)]; }
};
```

Coordinate compression and prefix sums

Source: include/core.hpp | SHA256 fragment 2ece16af

```
template<class T> std::vector<T> compressed_values(std::vector<T> a) {
    std::sort(a.begin(),a.end()); a.erase(std::unique(a.begin(),a.end()),a.end());
    return a;
}
template<class T> int rank_of(const std::vector<T>& sorted_unique, const T& x) {
    auto it = std::lower_bound(sorted_unique.begin(),sorted_unique.end(),x);
    return int(it-sorted_unique.begin());
}
inline std::vector<ll> prefix_sums(const std::vector<ll>& a) {
    std::vector<ll> p(a.size()+1);
    for (int i = 0; i < int(a.size()); ++i) p[i+1] = p[i]+a[i];
    return p;
}
```

Difference arrays

Source: include/core.hpp | SHA256 fragment 8962f751

```
inline void difference_add(std::vector<ll>& diff, int l, int r, ll value) {
    assert(0 <= l && l <= r && r < int(diff.size()));
    diff[l] += value; diff[r] -= value;
}
inline std::vector<ll> materialize_difference(const std::vector<ll>& diff) {
    assert(!diff.empty());
    std::vector<ll> a(diff.size()-1); ll cur = 0;
    for (int i = 0; i < int(a.size()); ++i) { cur += diff[i]; a[i] = cur; }
    return a;
}
```

Sliding window for nonnegative sums

Source: include/core.hpp | SHA256 fragment 8658ef6a

```
// Number of nonempty subarrays with sum <= limit; all a[i] must be >= 0.
inline ll count_subarrays_at_most(const std::vector<ll>& a, ll limit) {
    if (limit < 0) return 0;
    int l = 0; ll sum = 0, answer = 0;
    for (int r = 0; r < int(a.size()); ++r) {
        assert(a[r] >= 0); sum += a[r];
        while (sum > limit) sum -= a[l++];
        answer += r-l+1;
    }
    return answer;
}
```

Monotone binary search

Source: include/core.hpp | SHA256 fragment 0f09f243

```
// Predicate is false, ..., false, true, ..., true on [lo,hi).
template<class Pred> ll first_true(ll lo, ll hi, Pred ok) {
    assert(0 <= lo && lo <= hi);
    while (lo < hi) {
        ll mid = lo+(hi-lo)/2;
        if (ok(mid)) hi = mid; else lo = mid+1;
    }
    return lo; // Original hi if every tested position is false.
}
```

Fenwick sums and order statistics

Source: include/core.hpp | SHA256 fragment 3d06c1ce

```
struct Fenwick {
    int n; std::vector<ll> bit;
    explicit Fenwick(int n_) : n(n_), bit(n_+1) {}
    void add(int i, ll delta) {
        assert(0 <= i && i < n);
        for (++i; i <= n; i += i&-i) bit[i] += delta;
    }
    ll prefix(int r) const {
        assert(0 <= r && r <= n);
        ll answer = 0;
        for (; r; r -= r&-r) answer += bit[r];
        return answer;
    }
    ll sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        return prefix(r)-prefix(l);
    }
    // Positive, one-based target; all represented values must be nonnegative.
    int lower_bound(ll target) const {
        assert(target > 0);
        int pos = 0, step = 1; ll accumulated = 0;
        while (step <= n/2) step *= 2;
        for (; step; step /= 2) {
            int next = pos+step;
            if (next <= n && bit[next] < target-accumulated) {
                accumulated += bit[next]; pos = next;
            }
        }
        return pos; // n means no prefix reaches target.
    }
};
```

Iterative segment tree

Source: include/core.hpp | SHA256 fragment 76c48ac5

```
struct Sum { ll operator()(ll a,ll b) const { return a+b; } };
struct Minimum { ll operator()(ll a,ll b) const { return std::min(a,b); } };
template<class T, class Op> struct SegTree {
    int n, base = 1; T identity; Op op;
    std::vector<T> t;
    SegTree(const std::vector<T>& a, T identity_, Op op_ = Op())
        : n(int(a.size())), identity(identity_), op(op_) {
        while (base < n) base *= 2;
        t.assign(2*base,identity);
        for (int i = 0; i < n; ++i) t[base+i] = a[i];
        for (int i = base-1; i; --i) t[i] = op(t[2*i],t[2*i+1]);
    }
    void set(int i, T value) {
        assert(0 <= i && i < n);
        t[i += base] = value;
        while (i /= 2) t[i] = op(t[2*i],t[2*i+1]);
    }
    T query(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        T left = identity, right = identity;
        for (l += base,r += base; l < r; l /= 2,r /= 2) {
            if (l&1) left = op(left,t[l++]);
            if (r&1) right = op(t[--r],right);
        }
        return op(left,right);
    }
};
```

Capacity DP: 0-1 and unbounded knapsack

Source: include/core.hpp | SHA256 fragment 273e764b

```
// Max value with total weight <= capacity. Taking no items is allowed.
inline ll knapsack01(const std::vector<int>& weight,
                     const std::vector<ll>& value, int capacity) {
    assert(weight.size() == value.size() && capacity >= 0);
    std::vector<ll> dp(capacity+1);
    for (int i = 0; i < int(weight.size()); ++i) {
        assert(weight[i] >= 0);
        for (int c = capacity; c >= weight[i]; --c)
            dp[c] = std::max(dp[c],dp[c-weight[i]]+value[i]);
    }
    return dp[capacity];
}
inline ll knapsack_unbounded(const std::vector<int>& weight,
                             const std::vector<ll>& value, int capacity) {
    assert(weight.size() == value.size() && capacity >= 0);
    std::vector<ll> dp(capacity+1);
    for (int i = 0; i < int(weight.size()); ++i) {
        assert(weight[i] > 0);
        for (int c = weight[i]; c <= capacity; ++c)
            dp[c] = std::max(dp[c],dp[c-weight[i]]+value[i]);
    }
    return dp[capacity];
}
```

Strict LIS with reconstruction

Source: include/core.hpp | SHA256 fragment 0bed90fa

```
template<class T> std::vector<int> lis_indices(const std::vector<T>& a) {
    std::vector<T> tails;
    std::vector<int> ending, parent(a.size(),-1);
    for (int i = 0; i < int(a.size()); ++i) {
        auto it = std::lower_bound(tails.begin(),tails.end(),a[i]);
        int p = int(it-tails.begin());
        if (p > 0) parent[i] = ending[p-1];
        if (p == int(tails.size())) { tails.push_back(a[i]); ending.push_back(i); }
        else { tails[p] = a[i]; ending[p] = i; }
    }
    std::vector<int> result;
    if (!ending.empty())
        for (int i = ending.back(); i != -1; i = parent[i]) result.push_back(i);
    std::reverse(result.begin(),result.end()); return result;
}
} // namespace finals_core
```

Specialist lookup: choose the model before the template

This appendix is a source guide for the implementation handbook. Source snapshot: `7532cfe6c1eda7279d6e182b83d41e47e7760658` of pol-cova/kactl. Paths below start at `content/`. **P** means imported by the default chapter; **S** means present in source but omitted from that chapter. This describes the source configuration, not a claim that any previously built PDF contains it. Use the main handbook's basic graphs, arithmetic, and DP first.

KACTL headers assume the contest prelude (`ll`, `vi`, `pii`, `rep`, `all`, `sz`, standard includes). They are snippets, not independent libraries. Copy only what is needed and its dependencies. Global names such as `dfs`, `Time`, `mod`, `P`, and `conv` can collide. Use separate namespaces or adapt names when combining snippets. Source comments saying stress-tested describe upstream validation; the present handbook does not claim independent exhaustive verification of every header.

Fast lookup

Need	Header and exact entry point	Cost and main condition
All exact pattern occurrences	`strings/KMP.h`: `match(text, pattern)`	O(n+m); nonempty pattern; separator absent from inputs
Prefix borders	`strings/KMP.h`: `pi(s)`	O(n)
Prefix match at every suffix	`strings/Zfunc.h`: `Z(s)`	O(n); `z[0]=0`
Many fixed patterns	`strings/AhoCorasick.h`: `AhoCorasick ac(patterns)`	O(26L) build; default alphabet A-Z
Substring fingerprints	`strings/Hashing.h`: `HashInterval h(s)`	O(n) build, O(1) query; collisions possible
Palindrome radii	`strings/Manacher.h`: `manacher(s)`	O(n); separate even/odd conventions
Smallest cyclic rotation	`strings/MinRotation.h`: `minRotation(s)`	O(n); returns starting index
Sorted suffixes / adjacent LCP	`strings/SuffixArray.h`: `SuffixArray sa(s)`	O(n log n); includes empty suffix
Strong components of directed graph	`graph/SCC.h`: `scc(g, callback)`	O(V+E); recursive
Boolean pairwise clauses	`graph/2sat.h`: `TwoSat ts(n)`	O(V+E); literal negation is ~x
Undirected biconnected edge blocks	`graph/BiconnectedComponents.h`: `bicomps(callback)`	O(V+E); edge-ID adjacency
Bipartite maximum matching	`graph/HopcroftKarp.h`: `hopcroftKarp(g,r)`	O(E sqrt V); disjoint left/right indices
Bipartite minimum vertex cover	`graph/MinimumVertexCover.h`: `cover(g,n,m)`	matching-dependent; returns combined indices
Capacity flow / min cut	`graph/Dinic.h`: `Dinic f(n)` **S**	Header bound O(VE log U); residual capacities
Upward tree jumps / LCA	`graph/BinaryLifting.h`: `treeJump(parent)`	O(n log n) build, O(log n) query
Many static LCA queries	`graph/LCA.h`: `LCA lca(g)`	O(n log n) build, O(1) query; root 0
Tree path update / maximum	`graph/HLD.h`: `HLD<false> h(g)`	O(log² n); connected tree, root 0
Point predicates	`geometry/Point.h`: `Point<ll>`	O(1); bound intermediate products
Convex hull	`geometry/ConvexHull.h`: `convexHull(points)`	O(n log n); excludes intermediate collinear points
Segment intersection	`geometry/SegmentIntersection.h`: `segInter(a,b,c,d)`	O(1); integer output can truncate
General polygon containment	`geometry/InsidePolygon.h`: `inPolygon(poly,p,strict)`	O(n); simple polygon
Signed doubled polygon area	`geometry/PolygonArea.h`: `polygonArea2(poly)`	O(n); nonempty polygon
Convex polygon containment	`geometry/PointInsideHull.h`: `inHull(h,p,strict)`	O(log n); nonempty CCW hull, no collinear vertices
Farthest points	`geometry/HullDiameter.h`: `hullDiameter(h)`	O(n) after hull; nonempty hull
Closest points	`geometry/ClosestPair.h`: `closest(points)`	O(n log n); at least two points
Prefix count in a sweep	`data-structures/FenwickTree.h`: `FT f(n)`	O(log n); zero indexed, half-open prefix

Need	Header and exact entry point	Cost and main condition
Offline 2D count	`data-structures/FenwickTree2d.h`: `FT2 f(nx)`	O(log² n); register updates before init
Offline interval queries	`data-structures/MoQueries.h`: `mo(queries)`	O(n sqrt q) moves; callbacks must be written
Max over lines / DP optimization	`data-structures/LineContainer.h`: `LineContainer lc`	O(log n) amortized insert/query; integer maxima
Fixed-size matrix power	`data-structures/Matrix.h`: `Matrix<T,N>`	O(N³ log exponent); no automatic modulus
Polynomial convolution	`numerical/FastFourierTransform.h`: `conv(a,b)`	O(N log N); precision bound below
Modular convolution	`numerical/NumberTheoreticTransform.h`: `conv(a,b)`	O(N log N); fixed supported prime and length
Huge index linear recurrence	`numerical/LinearRecurrence.h`: `linearRec(S,tr,k)`	O(d² log k); configure modulus explicitly

All rows are P unless marked S. For a header's transitive dependencies, follow its actual `#include` statements. The sections below explain the decisions that a lookup table cannot make.

Strings: structure before hashing

KMP and Z. `auto positions = match(text, pattern);` returns zero-based starts, including overlaps. `pi(s)[i]` is the longest proper prefix ending at i. A candidate period is `p=n-pi(s).back()`; full repetition requires `n%p==0`. A border alone does not imply an integral number of repeats. `auto z=Z(s);` gives the common-prefix length of `s` and `s.substr(i)`, with z[0]=0. For matching via `pattern + separator + text`, choose a character absent from both alphabets. KMP's built-in `match` uses nul; ordinary token strings satisfy this, arbitrary byte buffers may not. Treat empty patterns separately. Both use O(n) extra memory.

Trie / Aho-Corasick. There is no separate general-purpose `Trie.h` in this snapshot. Aho-Corasick builds a trie, then fills missing transitions through failure links; afterwards its `next` array is an automaton, not a sparse trie whose absent edges remain -1. Default `first='A'`, `alpha=26`: lowercase input without adaptation indexes outside the array. Construct from a mutable vector of nonempty patterns:

```
vector<string> patterns = {"AB", "BAB"};
AhoCorasick ac(patterns);
auto endings = ac.find("ABAB");
auto starts = ac.findAll(patterns, "ABAB");
```

`find` returns one pattern index per text position, the longest ending there or -1. `findAll` returns all matching pattern indices grouped by start position, including duplicates. Do not mistake `find` for all matches. Build O(alpha*total pattern length), scan O(text length); enumerating all matches additionally costs the output size, which can be quadratic. The source header gives a coarse O(NM) bound for `findAll`.

Hashing. `HashInterval h(s); auto fingerprint=h.hashInterval(l,r);` hashes [l,r) in O(1), preprocessing and memory O(n). Compare lengths as well as hashes. This header uses arithmetic modulo 2^64-1, not the simpler overflow hash modulo 2^64. Hash equality is evidence, never a deterministic proof of equality. Prefer KMP/Z for exact matching; use hashes when many arbitrary substring comparisons make their convenience worthwhile and collision risk is acceptable. Do not mix hash objects produced with different bases or alphabets. `Hashing-codeforces.h` is a separate source variant, not the chapter import.

Other string APIs. `auto p=manacher(s)` gives even radius p[0][i] and odd half-length p[1][i]; lengths are `2*p[0][i]` and `2*p[1][i]+1`. Test centers on "abba" and "aba" before using indices. `int i=minRotation(s)` selects the lexicographically smallest rotation; rotate using `rotate(s.begin(),s.begin()+i,s.end())`. `SuffixArray a(s)` contains n+1 suffixes, with `a.sa[0]`=n, the empty suffix. `a.lcp[i]` compares suffixes sa[i] and sa[i-1]. No nul bytes; ordinary nonnegative ASCII avoids signed-char alphabet issues. Arbitrary two-suffix LCP requires inverse ranks plus RMQ on the appropriate LCP interval, not direct indexing by text position.

Directed constraints and undirected connectivity

SCC. With `vector<vi> g`, call `scc(g,[&](vi& block){ /* consume vertices */ });`. Afterwards `comp[v]` and `ncomps` are globals. Components are emitted in reverse topological order: every inter-component edge goes from a larger component ID to a smaller one. Build the condensation DAG only after all IDs exist; deduplicate edges if multiplicity affects later counting. O(V+E) time and memory. Recursive DFS can exhaust the process stack on a long chain; stack feasibility must be addressed for large constraints. The callback's vector is reused; copy it if storing blocks. `dfs`, `Time`, and other globals collide with other snippets if blindly pasted.

2-SAT. A variable is its zero-based integer x; its negation is `~x`, never `!x` or `-x`.

```
TwoSat ts(3);
ts.either(0, ~1);       // x0 OR NOT x1
// x1 => x2 is NOT x1 OR x2:
ts.either(~1, 2);
ts.setValue(0);
if (ts.solve()) { int x2 = ts.values[2]; }
```

Clauses must reduce to ORs of at most two literals. A requirement that exactly one of k>2 arbitrary variables holds is not generally captured by `atMostOne` alone: that method supplies only the upper bound. `addVar()` introduces auxiliaries; values includes them. Solve O(number of variables + implication edges). It has its own SCC implementation; it does not include `SCC.h`. Reconstruct the actual modeling choice from `values`, and check every original constraint on a tiny example.

Biconnected blocks and bridges. Set global `ed` to n empty adjacency lists, insert each undirected edge twice with one shared unique edge ID, then call `bicomps(callback)`. The callback receives edge IDs belonging to a non-bridge biconnected block. A vertex can belong to multiple blocks; a bridge is detected in the explicit `/* e is a bridge */` branch and is not automatically returned. To collect bridges, edit that branch. Self-loops are not a supported block-reporting convention here; handle them separately if allowed. Parallel edges are distinguished correctly only if each physical edge has its own ID. Skipping the parent vertex instead of the parent edge would break this property. Articulation points need additional bookkeeping, including the root-child rule. Do not interpret these blocks as SCCs or as a vertex partition.

Matching and capacity models

Hopcroft-Karp. g has one list per left vertex; entries are right-side indices [0,m). Initialize `vi rightMatch(m,-1); int size=hopcroftKarp(g,rightMatch);`. It returns cardinality and fills rightMatch[right]=left, or -1. The left match can be reconstructed by scanning this array. O(E sqrt V), O(V+E) memory. It solves unweighted bipartite matching, not general graphs, weights, or capacity>1 directly. Bipartiteness belongs to the model: separate object types do not automatically imply all constraints are pairwise compatibility edges. `cover(g,n,m)` from `MinimumVertexCover.h` returns left IDs [0,n) and right IDs shifted by n. Its size equals maximum matching cardinality; it includes and uses DFSMatching, not HopcroftKarp.

Dinic. Source-only by default: explicitly include or print it if selecting it for contest use.

```
Dinic flow(n);
flow.addEdge(s, a, 3); // directed capacity 3, default reverse capacity 0
flow.addEdge(a, t, 2);
ll sent=flow.calc(s,t);
bool sourceSide=flow.leftOfMinCut(a); // after completed calc
```

Capacities must be nonnegative integers fitting `ll`; source and sink must differ. `calc` mutates residual capacities and returns additional flow sent by that call. `Edge.c` is remaining capacity, `oc` original capacity, `flow()` reports nonnegative net usage for that directed edge. Keep an adjacency index when you need to inspect a specific modeled edge. `leftOfMinCut(v)` reports final residual reachability after max flow; it is not valid before calc. The header advertises O(VE log U), and faster unit-capacity/bipartite bounds. Memory O(V+E). DFS is recursive.

Matching model: source -> each left vertex capacity1; allowed left -> right capacity1; right -> sink capacity1. Vertex capacity: split v into v_in -> v_out and route incident edges through the split. Selection with penalties: derive which cut direction charges each forbidden choice before coding. "Infinite" means greater than every feasible finite cut, with a proved safe total bound; LLONG_MAX edges can make total flow overflow. `rcap` supplies initial reverse capacity, not a second independent labeled object. Avoid self-loops in this header's addEdge unless adapting reverse-index bookkeeping. Parallel non-self edges are fine. Flow does not solve min-cost optimization; that needs the separate MinCostMaxFlow API and its cost assumptions.

Trees: path conventions are part of the algorithm

Binary lifting. Supply parent[root]=root and valid parent indices, with depths computed separately. `auto up=treeJump(parent); int a=jmp(up,v,k); int w=lca(up,depth,u,v);`. O(n log n) build/storage, O(log n) query. These tables are for upward ancestors, not arbitrary shortest paths. Ancestor steps beyond represented bits are not checked; restrict k to the supported range or enlarge the table. The RMQ-based `LCA tree(g); tree.lca(u,v);` instead roots at0 and gives O(1) queries after O(n log n) preprocessing; requires its RMQ dependency and recursive traversal.

HLD. `HLD<false> h(g)` stores vertex values; `HLD<true>` stores each edge at its deeper endpoint. Input must be a nonempty connected undirected tree rooted at0. The class copies adjacency and deletes parents from its private copy. `modifyPath(u,v,x)` adds x along the path; `queryPath(u,v)` returns its maximum. `querySubtree(v)` uses the contiguous Euler interval [pos[v],pos[v]+siz[v]), excluding v in edge mode. O(log² n) path operations, O(log n) subtree query with its lazy tree. The source includes `LazySegmentTree.h`, whose initial values are its negative default, not zero: initialize each relevant position before arithmetic updates, for example `h.tree->set(h.pos[v],h.pos[v]+1,initial[v])`. Path and subtree ranges are half open. For edge mode, u=v is an empty path and should have a deliberately chosen answer. Combining path pieces in this order assumes a commutative aggregate; string concatenation or matrix products require direction-aware processing. Change both the underlying segment tree and the HLD aggregation when switching max to sum/min.

Geometry: distinguish predicates from constructed coordinates

Use `Point<ll>` for exact integer dot/cross predicates only when bounds fit. For coordinates |x|,|y|<=B, differences reach2B and a difference cross product can reach8B². Casting the final result is too late: widen before multiplication if using `__int128`. `Point<double>` is appropriate for constructed fractional coordinates; unit vectors and rotation should not use integer points because division truncates. A single arbitrary epsilon does not fix every geometric comparison.

`a.cross(b,c)` is (b-a)*(c-a), positive for a counterclockwise turn; `u.dot(v)` distinguishes acute/obtuse directions. Use squared distances for ordering when safe. `onSegment(a,b,p)` includes endpoints. Test a segment touching at an endpoint, overlapping collinearly, degenerate to a point, and disjoint.

`auto h=convexHull(points)` returns a CCW hull, removing intermediate collinear boundary points. Empty or all-collinear input produces a degenerate hull; handle its size before later operations. `hullDiameter(h)` assumes nonempty, CCW, no duplicate/collinear vertices and finds the farthest pair in O(h). `closest(points)` requires at least two points. Their squared distance arithmetic also needs overflow bounds.

`auto inter=segInter(a,b,c,d)` returns zero points for no intersection, one for a point, two endpoints for overlapping segments. Its proper intersection formula uses triple-coordinate products; with `Point<ll>` a nonintegral intersection is truncated. For existence alone, prefer orientation/sign predicates without constructing coordinates. For coordinate output, use an appropriate floating/rational representation and the problem's tolerance. `inPolygon(poly,p,false)` includes boundary, default strict=true excludes it; requires a simple polygon and O(n) per query. `inHull(h,p,false)` needs a nonempty CCW convex hull with collinear interior-edge points removed, O(log h); the implementation dereferences index0 even for an empty hull. `polygonArea2(poly)` returns signed doubled area, positive CCW, negative clockwise; never call on an empty vector. Keep doubled area integral for half-unit answers.

Sweeps and optimized transitions

Sort before reaching for a tree. For offline requests "count points with x<=X and y<=Y", sort by x and coordinate-compress y, update FT for active points, answer prefix counts. Handle equality by explicit event order: insert equal-x points before queries for <=, afterwards for <. `FT.update(i,delta)` adds; `FT.query(r)` returns [0,r). Thus [l,r) is query(r)-query(l). O((n+q)log n). Compression preserves order/equality, not coordinate distances. A DP with strict predecessor coordinates must delay updates for an equal-coordinate batch to avoid using peers from the same layer.

`FT2 f(nx)` is for update points known offline: call fakeUpdate(x,y) for every possible location, then init() once, then update(x,y,delta). `query(X,Y)` sums x<X,y<Y; rectangle sums use four prefixes. Registration is mandatory even if updates occur later. x is already a valid dense index. Memory O(n log n) for n registered points; query/update O(log² n). `FT.lower_bound(target)` is an order-statistic search requiring nonnegative element frequencies / monotone prefix sums; it is not valid after arbitrary negative updates. Its literal initial search bit `1<<25` also limits the directly supported size.

Mo. Use for offline static intervals when add/remove can update the answer cheaply but no small associative segment summary exists. `MoQueries.h` deliberately contains `...`: implement add, del, calc first; it is not copy-and-run code. `mo(Q)` uses [l,r), starts empty at [0,0), and returns answers in original query order. Its O(n sqrt q) movement estimate assumes suitable block size and O(1) callbacks; source blk=350 is a tuning constant, not universal. A frequency-of-frequencies example is a good rehearsal; dynamic updates require a different algorithm.

Line container. For `dp[i]=max_j(k[j]*x[i]+b[j])+own[i]`, insert a line for each eligible j, then query x[i]. `LineContainer lc; lc.add(k,b); ll best=lc.query(x);`. O(log n) amortized add/query; source handles arbitrary insertion/query order, duplicate slopes, integer breakpoints via floor division. Empty query asserts. For minimum, negate both slope/intercept and negate the answer, with overflow checked. This is valid only after deriving a linear transition. Products k*x and differences of slopes/intercepts must fit ll; enlarging the answer type alone does not fix internal overflow. Time eligibility still matters: insert before or after query according to whether j=i is legal.

Matrices and transforms: specialist fallback

`Matrix<T,N> A;` is zero-initialized. Fill A.d; `(A^k)*vector` applies k transitions to a column state. If state_next[i]=sum_j A[i][j]*state[j], rows are destinations and columns sources. Cost O(N³ log k), memory O(N²); use only small state dimensions. Ordinary `Matrix<ll,N>` does not reduce modulo anything and readily overflows. Use a verified modular numeric type or adapt multiply/add with safe intermediate products. Exponent0 returns identity. For small k, ordinary DP is simpler.

`linearRec(S,tr,k)` computes S[k] from initial S[0..d-1] and coefficients `S[i]=sum_j S[i-j-1]*tr[j]`, in O(d² log k). Source contains `const ll mod=5` marked `exclude-line`; that is a source/testing placeholder, not the intended contest modulus. Configure mod explicitly when copying, normalize residues, and check multiplication bounds. Berlekamp-Massey can infer a recurrence over a suitable field but does not prove an observed prefix follows it forever.

FFT `conv(vector<double>,vector<double>)` returns coefficients c[k]=sum_i a[i]b[k-i], O(N log N) at padded power-of-two length N. Manual fft requires positive power-of-two length. For rounded integer convolution the source's sufficient bound is `(sum a[i]^2+sum b[i]^2)*log2(N)<9e14`; do not promise exactness above it. It returns doubles: explicit rounding is needed for integer counts. NTT's `conv(vector<ll>,vector<ll>)` instead returns exact coefficients modulo998244353, requires normalized inputs [0,mod), and depends on ModPow.h. The implementation chooses a power of two strictly larger than the requested output size, so require that actual padded N<=2^23, not just output size<=2^23. FFT and NTT define overlapping global symbols: select one or namespace them. Modular convolution is not integer convolution unless you prove coefficients fit the modulus or reconstruct with sufficient independent primes.

Printed source pack

These are selected original source expansions, not a single file to concatenate indiscriminately. Keep needed dependencies once; put SCC's globals/functions in their own namespace when combining with other DFS snippets. Geometry's repeated `Point.h` includes become one printed definition; `#pragma once` handles source includes. The selected pack needs the following GNU C++17 prelude (SCC/2-SAT use GNU's omitted-middle conditional `a ?: b`):

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i,a,b) for (int i=(a); i<(b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using vi = vector<int>;
using pii = pair<int,int>;
```

For standalone submissions add main, input parsing, output, and problem-specific model construction. Read each header's alphabet, indexing, boundary, and type conditions above before copying.

Exact one-pattern matching

KACTL code: KMP

Source: ../content/strings/KMP.h | Author: Johan Sannemo | License: CC0

```
#pragma once

vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

Prefix matches

KACTL code: Zfunc

Source: ../content/strings/Zfunc.h | Author: chilli | License: CC0

```
#pragma once

vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Many-pattern automaton

KACTL code: AhoCorasick

Source: ../content/strings/AhoCorasick.h | Author: Simon Lindholm | License: CC0

```
#pragma once

struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
```

```
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

Directed strong components

KACTL code: SCC

Source: ../content/graph/SCC.h | Author: Lukas Polacek | License: CC0

```
#pragma once

vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

Pairwise Boolean constraints

KACTL code: 2sat

Source: ../content/graph/2sat.h | Author: Emil Lenngren, Simon Lindholm | License: CC0

```
#pragma once

struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

Capacity flow and residual cut

KACTL code: Dinic

Source: ../content/graph/Dinic.h | Author: chilli | License: CC0

```
#pragma once

struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

Bipartite cardinality matching

KACTL code: HopcroftKarp

Source: ../content/graph/HopcroftKarp.h | Author: Adam Soltan | License: CC0

```
#pragma once

int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

Basic tree ancestors

KACTL code: BinaryLifting

Source: ../content/graph/BinaryLifting.h | Author: Johan Sannemo | License: CC0

```
#pragma once

vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

Exact point operations

KACTL code: Point

Source: ../content/geometry/Point.h | Author: Ulf Lundstrom | License: CC0

```
#pragma once

template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

Segment membership dependency

KACTL code: OnSegment

Source: ../content/geometry/OnSegment.h | Author: Victor Lecomte, chilli | License: CC0

```
#pragma once

#include "Point.h"

template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

Segment intersection construction

KACTL code: SegmentIntersection

Source: ../content/geometry/SegmentIntersection.h | Author: Victor Lecomte, chilli | License: CC0

```
#pragma once

#include "Point.h"
#include "OnSegment.h"

template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

Convex hull construction

KACTL code: ConvexHull

Source: ../content/geometry/ConvexHull.h | Author: Stjepan Glavina, chilli | License: Unlicense

```
#pragma once

#include "Point.h"

typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

Signed doubled area

KACTL code: PolygonArea

Source: ../content/geometry/PolygonArea.h | Author: Ulf Lundstrom | License: CC0

```
#pragma once

#include "Point.h"

template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

Dynamic linear transition maximum

KACTL code: LineContainer

Source: ../content/data-structures/LineContainer.h | Author: Simon Lindholm | License: CC0

```
#pragma once

struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

Fixed-state matrix powers

KACTL code: Matrix

Source: ../content/data-structures/Matrix.h | Author: Ulf Lundstrom | License: CC0

```
#pragma once

template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```